

CROSSY ROAD

C++ Terminal Edition

Complete System Documentation
Pseudo-3D Isometric ASCII Renderer
Linked-List Architecture
Persistent Leaderboard System
Player Name Support

Authors

Dimabayao, Karl Ancel T.
Ramos, Timothy Christopher A.
Eborde, Benedict Ronald D.
Alvarez, Rob Angel S.
Abillon, Kent Steven N.

To

Sir Lastimoso, Ian Peter L.

Date

04/30/2026

Ateneo De Naga University
BS In Information System
BS In Information Technology

Table of Contents

Table of Contents.....	2
1. Program Overview.....	3
2. Features & Functionality.....	3
2.1 Core Gameplay.....	3
2.2 3D Isometric Renderer.....	4
2.3 Player Name System.....	5
2.4 Leaderboard System.....	5
3. Installation & Setup.....	5
3.1 Prerequisites.....	5
3.2 Compilation.....	5
3.3 Running the Game.....	6
4. How the Leaderboard Works.....	6
4.1 Data Storage Format.....	6
4.2 Score Submission Flow.....	7
4.3 Viewing the Leaderboard.....	7
4.4 Sample Leaderboard Display.....	8
5. Example Usage.....	8
5.1 A Full Session Walk-Through.....	8
5.2 Resetting the Leaderboard.....	8
6. Key Code Components.....	9
6.1 Code Structure Overview.....	9
6.2 Leaderboard Core Functions.....	10
loadLeaderboard().....	10
submitScore(name, score).....	10
6.3 World & Lane Management.....	11
6.4 3D Rendering Pipeline.....	11
6.5 Physics Model.....	12
6.6 Terminal Raw Mode.....	12
6.7 Unicode Half-Block Rendering.....	12
7. Configuration Reference.....	13
8. Troubleshooting.....	14
9. Flow Chart.....	14

1. Program Overview

Crossy Road 3D (C++ Terminal Edition v4.0) is a feature-rich terminal game inspired by the mobile hit Crossy Road. Written entirely in standard C++17, it runs in any POSIX-compatible terminal that supports 256-color ANSI escape codes. The player controls a chicken character navigating an infinitely scrolling isometric landscape of roads, rivers, and grasslands.

The v4.0 release adds three major capabilities to the original rendering engine: a player name prompt at startup, a persistent CSV-backed leaderboard that ranks scores across all sessions, and an in-game leaderboard viewer accessible at any time by pressing L.

 **Design Philosophy:** The project deliberately avoids external game libraries (SDL, ncurses, etc.). Every pixel, color, and input event is handled through raw ANSI escape codes and POSIX terminal APIs, making it a self-contained study in low-level systems programming and data structures.

2. Features & Functionality

2.1 Core Gameplay

● Infinite procedural world

Lanes are generated on demand and garbage-collected behind the player.

● Four terrain types

Safe (cobblestone), Grass, Road (cars), and Water (logs).

● Physics

Cars kill on contact. Falling off a log into water is fatal. Logs carry the player downstream.

● Score

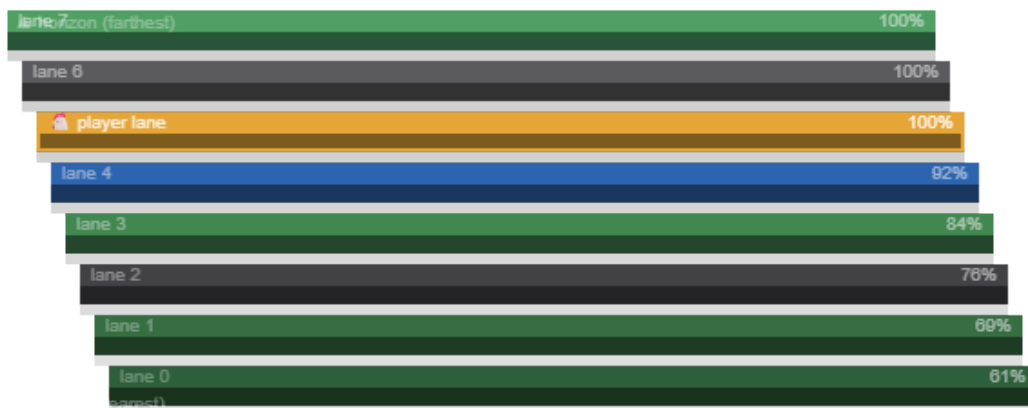
Equals the highest absolute lane index reached in a session.

● Jump animation

The player character flashes when moving.

2.2 3D Isometric Renderer

Each lane renders as 3 terminal rows	Top face (■), front face (■), and a drop shadow (■) – using Unicode half-block characters.
Isometric perspective	Lanes shift one column right per depth step, producing a convincing fake-3D look without a real 3D engine.
Depth-aware shading	Colors darken smoothly with distance using a fade factor (100% at player level → ~45% at the horizon).
Full double-buffer	The entire screen is built in memory and flushed atomically to minimize flicker.



2.3 Player Name System

- At startup, the splash screen prompts the player to type their name (max 10 characters).
- The name appears in the HUD during gameplay and in all leaderboard entries.
- Empty input defaults to the name "Anonymous".

2.4 Leaderboard System

- Scores are saved to `leaderboard.csv` in the working directory after every session.
- The file stores one entry per line in the format: `name, score`.
- Entries are sorted in descending order by score; only the top 10 are retained.
- The leaderboard displays on the splash screen, can be viewed in-game (L key), and shows again after the session ends.
- Rank is calculated and shown in the end-of-game summary box.

3. Installation & Setup

3.1 Prerequisites

Requirement	Details
Operating System	Linux or macOS (POSIX terminal required)
Compiler	<code>g++ 7+</code> or <code>clang++ 5+</code> with C++17 support
Terminal	Any 256-color terminal (xterm, iTerm2, GNOME Terminal, etc.)
Terminal size	Minimum 80 columns × 45 rows recommended
Disk space	< 1 MB (source + binary + CSV)

3.2 Compilation

Save the source file as `crossy_road.cpp`, then compile with:

```
# Recommended: optimized release build
g++ -std=c++17 -O2 -Wall -Wextra -o crossy_road crossy_road.cpp

# Debug build (with sanitizers)
g++ -std=c++17 -g -fsanitize=address,undefined -o crossy_road_dbg
crossy_road.cpp

# macOS with clang (identical flags)
clang++ -std=c++17 -O2 -Wall -o crossy_road crossy_road.cpp
```

Zero-dependency: The program uses only the C++ standard library plus POSIX headers (<termios.h>, <unistd.h>, <csignal>). No external packages, build systems, or package managers are required.

3.3 Running the Game

```
1st Run in the current directory (leaderboard.csv is created
here)
./crossy_road
2nd The splash screen appears first.
3rd Type your name and press Enter to begin.
```

4. How the Leaderboard Works

4.1 Data Storage Format

Scores are persisted in a plain-text CSV file named `leaderboard.csv` located in the same directory as the executable. Each line contains exactly one entry:

```
# leaderboard.csv format
PlayerName, Score

# Example file contents:
Alice,142
Bob,97
```

Charlie,84
Anonymous,31

Commas within player names are supported because the file is parsed by splitting on the last comma on each line, not the first. Names may contain any character except a leading/trailing newline.

4.2 Score Submission Flow

1. At session end, `submitScore(name, score)` is called.
2. The existing `leaderboard.csv` is loaded into a `vector<LeaderboardEntry>`.
3. The new entry is appended to the vector.
4. The vector is sorted in descending order by score using `std::sort` with a lambda comparator.
5. The rank of the new entry is recorded (its 1-based position after sorting).
6. The vector is trimmed to `MAX_LEADERBOARD_ENTRIES` (10) entries.
7. The trimmed vector is written back to `leaderboard.csv`, overwriting the old file.

4.3 Viewing the Leaderboard

When	How
Splash screen	Automatically shown before the name prompt
In-game (alive)	Press L – game pauses, board shown, Enter to resume
After death	Press L – same overlay before restarting or quitting
Session end	Automatically shown with rank highlight

4.4 Sample Leaderboard Display

🏆 LEADERBOARD 🏆		
RANK	PLAYER	SCORE
1	Alice	142
2	Bob	97
3	Charlie	84

5. Example Usage

5.1 A Full Session Walk-Through

8. Compile and run `./crossy_road` in the terminal.
9. The splash screen displays the ASCII title, controls, current leaderboard, and a name prompt.
10. Type your name (e.g., "Alice") and press Enter.
11. The game starts. Move with WASD or arrow keys – cross roads, hop logs, avoid drowning.
12. Press L at any time to see the leaderboard overlay, then Enter to resume.
13. When the player dies, the footer shows GAME OVER with options to restart (R), view the board (L), or quit (Q).
14. Press Q. The session summary shows your score, best, and rank (e.g., "Rank: #2 on leaderboard").
15. The updated leaderboard prints automatically. Your entry now appears sorted by score.

5.2 Resetting the Leaderboard

To clear all scores, simply delete or overwrite the CSV file:

```
rm leaderboard.csv      # Delete entirely (file recreated on next
run)
> leaderboard.csv       # Truncate to empty (same effect)
```

6. Key Code Components

6.1 Code Structure Overview

The source file is divided into 19 clearly labelled sections, each handling one logical concern:

Section	Name	Responsibility
1	Terminal Helpers	Raw mode, cursor control, ANSI color sequences
2	Game Constants	Board dimensions, timing values, and leaderboard settings
3	Leaderboard System	load/save CSV, submitScore, displayLeaderboard
4	Visual Palette	Depth-aware color tables per lane type
5	Linked List	Generic SLinkedList<T> template for obstacles
6	Obstacle	Car/log data: position, width, speed, visual type
7	Lane	Terrain strip with obstacle list; doubly-linked
8	World	Doubly-linked lane list; O(n) laneAt lookup
9	RNG Helpers	Integer and floating-point random helpers
10	Lane Factory	makeLane(), buildInitialWorld(), ensureLanes()

Section	Name	Responsibility
11	Physics	tickLane(), findObs(), physTick(), tryMove()
12	Player	Position, score, animation state
13	3D Render Engine	ScreenBuf, TileInfo, drawLane3D()
14	HUD & Footer	drawHUD3D(), drawFooter3D(), drawDepthRail()
15	Full Frame Render	render3D() – assembles and flushes one frame
16	Input Handling	pollKey(), Key enum
17	Splash Screen	splash() – title art, controls, name prompt
18	Game Loop	runGame() – main event/physics/render loop
19	Entry Point	main(), signal handler cleanup()

6.2 Leaderboard Core Functions

loadLeaderboard()

Opens leaderboard.csv and parses it line by line. Each line is split on the last comma to extract the name and score. Malformed lines are silently skipped. Returns a `std::vector<LeaderboardEntry>`.

```
std::vector<LeaderboardEntry> loadLeaderboard() {
    std::ifstream file(LEADERBOARD_FILE);
    // Parse: split each line on last comma → name + score
    auto pos = line.rfind(',');
    entries.push_back({ line.substr(0, pos),
stoi(line.substr(pos+1)) });
}
```

submitScore(name, score)

Loads existing entries, appends the new one, sorts descending, records rank, trims to MAX_LEADERBOARD_ENTRIES, and saves. Returns the 1-based rank of the submitted entry, or -1 if it didn't make the cut.

```
int submitScore(const std::string& name, int score) {
    auto entries = loadLeaderboard();
    entries.push_back({name, score});
    std::sort(entries.begin(), entries.end(),
        [](const auto& a, const auto& b){ return a.score >
b.score; });
    // find rank, trim to MAX_LEADERBOARD_ENTRIES, save
}
```

6.3 World & Lane Management

The World struct maintains a doubly-linked list of Lane objects. Each Lane has a prev/next pointer forming the chain. The head is the farthest (highest absY) lane; the tail is the nearest (lowest absY). As the player advances, new lanes are prepended to the head via ensureLanes() and old lanes are deleted from the tail via popBottom().

laneAt(absY) walks from whichever end is closer to the requested position, giving $O(n/2)$ average access with n = visible window (~30 lanes typically). This is fast enough for the 20fps render loop.

6.4 3D Rendering Pipeline

Each call to render3D() proceeds in this order:

16. Allocate ScreenBuf – a 2D grid of ScreenCell structs, each holding a full ANSI escape sequence string.
17. Flood-fill background with a dark gray (#232) cell.
18. drawHUD3D() – writes the title/score bar into rows 0-2 of the buffer.
19. For each of the BH=12 visible screen lanes: call drawLane3D(), which fills 3 buffer rows (top face, front face, shadow) for that lane's terrain, obstacles, and player.
20. drawDepthRail() – draws isometric edge pillars on the left margin.

21. `drawFooter3D()` – writes controls or GAME OVER into the last 2 rows.
22. `ScreenBuf::flush()` – concatenates all cells into one string and writes it to stdout with a single move-to-home sequence, minimizing terminal redraws.

6.5 Physics Model

The game runs on two independent timers – a physics tick (90 ms) and a render tick (50 ms):

Timer	Behavior
TICK_MS = 90 ms	All obstacles in nearby lanes advance by their speed. Log drift is applied to the player if on water.
FRAME_MS = 50 ms	<code>render3D()</code> is called to flush a new frame. ~20 FPS.
Input poll	Checked every loop iteration (~8 ms sleep). Movement is applied immediately on keypress.

6.6 Terminal Raw Mode

- The terminal starts in normal mode (line buffering + echo on) and transitions to raw mode via `Term::raw()` at game start
- `ICANON=0` and `ECHO=0` are shown as the two flags that raw mode clears
- `VMIN=0` + `VTIME=0` together enable the non-blocking `read()` that powers the poll loop
- The saved `termios` struct captures the original settings at startup
- Both exit paths – clean quit and `SIGINT` (Ctrl+C) via the `cleanup` handler – both flow back to `Term::restore()`, guaranteeing the terminal is always recovered

6.7 Unicode Half-Block Rendering

The 3D voxel appearance is achieved with just four Unicode characters, combined with 256-color foreground and background pairs:

Char	UTF-8	Usage
■	\xe2\x96\x84	Top face of terrain/obstacle tiles (lower half filled)
▀	\xe2\x96\x88	Front face – solid color blocks
▏	\xe2\x96\x8c	Isometric depth rail on the left edge
▒	\xe2\x96\x91-93	Ground shadow, water ripple, bark texture

7. Configuration Reference

All tuneable constants are defined at the top of the source file. Edit and recompile to change game behaviour:

Constant	Default	Description
BW	40	Board width in tiles
BH	12	Number of visible lanes
LANE_H	3	Terminal rows per lane
ISO_SKEW	1	Isometric column shift per depth step
PLAYER_LANE	BH-3	Player's fixed screen slot (0=top)
AHEAD_BUFFER	BH+4	Lanes are pre-generated ahead of the player
BELOW_BUFFER	BH+4	Lanes are retained behind the player
TICK_MS	90	Physics update interval (ms)
FRAME_MS	50	Render interval (ms, ~20 fps)
LEADERBOARD_FILE	leaderboard.csv	CSV file path for scores

Constant	Default	Description
MAX_LEADERBOARD_ENTRIES	10	Maximum entries retained in the leaderboard

8. Troubleshooting

Problem	Solution
Garbled characters/boxes	Ensure your terminal uses a UTF-8 locale: <code>export LANG=en_US.UTF-8</code>
Wrong colors/monochrome	Use a 256-color terminal. Check: <code>echo \$TERM</code> should show <code>xterm-256color</code>
Terminal doesn't restore after a crash	Run: <code>reset</code> (or <code>stty sane</code>) to restore terminal state
Leaderboard file not created	Check write permissions in the working directory
Name prompt not shown	Ensure stdin is a real terminal (not piped/redirected)
Flickering on slow machines	Increase <code>FRAME_MS</code> to 20-30 to reduce render rate

9. Flow Chart

